School of Informatics & Cybersecurity
Assessment Submission Coversheet

Student Information

Student Number:	[redacted]
Student Name:	Maël Conejos Crusson

Module Information

Module Title:	Security Administration
Module Code:	[redacted]

Assessment Information

Assessment Title:	Security Administration Project

Declaration

☑ I confirm that I have read and understood the University's Academic Integrity Policy.

[redacted]

☑ I have completed the TU Dublin AI Driving License theory test.

[redacted]

☑ I declare that this assessment submission is my own work.

☑ I confirm that I have read and understood the AI traffic light system used as part of this assessment brief and I have adhered to the level of AI usage allowed for this assessment.

Student signature:	[redacted]
Date:	07/12/2025

[redacted]

Security Administration Project

TechStart Solutions

[redacted]aël Conejos Crusson

07/12/2025

Table of Contents

Summary 3
Architecture Overview 3
Mandatory Security Requirements 10
Requirement 1: Network Segmentation & Access Control 10
Requirement 2: Strong Authentication & Access Management 12
Requirement 3: Encryption in Transit and at Rest 16
Requirement 4: System Hardening & Monitoring 17
BASH Scripts 19
Requirement 5: Logging, Backup & Audit Trail 22
Additional Security Measures 24
Access Control Matrix 27
App Deployment 28
Conclusion 38

Summary

This project has to do with the creation of TechStart Solutions, a fictional organisation with which its web application is deployed in a secure cloud environment created by me. TechStart Solutions is a company with 6 employees focusing on software development, with staff such as developers, QA testers, and project managers.

The primary components used in this project are a virtual machine, an SQL database, and Blob storage. The entire infrastructure of the system follows to the Principle of Least Privilege and Defence in Depth. This means that the staff only have access strictly to what they need and security measures are implemented to secure the infrastructure.

Architecture Overview

All resources used for this project are stored in a resource group named 'techstart_security_rg' (Fig. 1), which is a logical container for resources that are related to each other. The resource group allows me to manage, deploy, and delete all resources as a single unit.

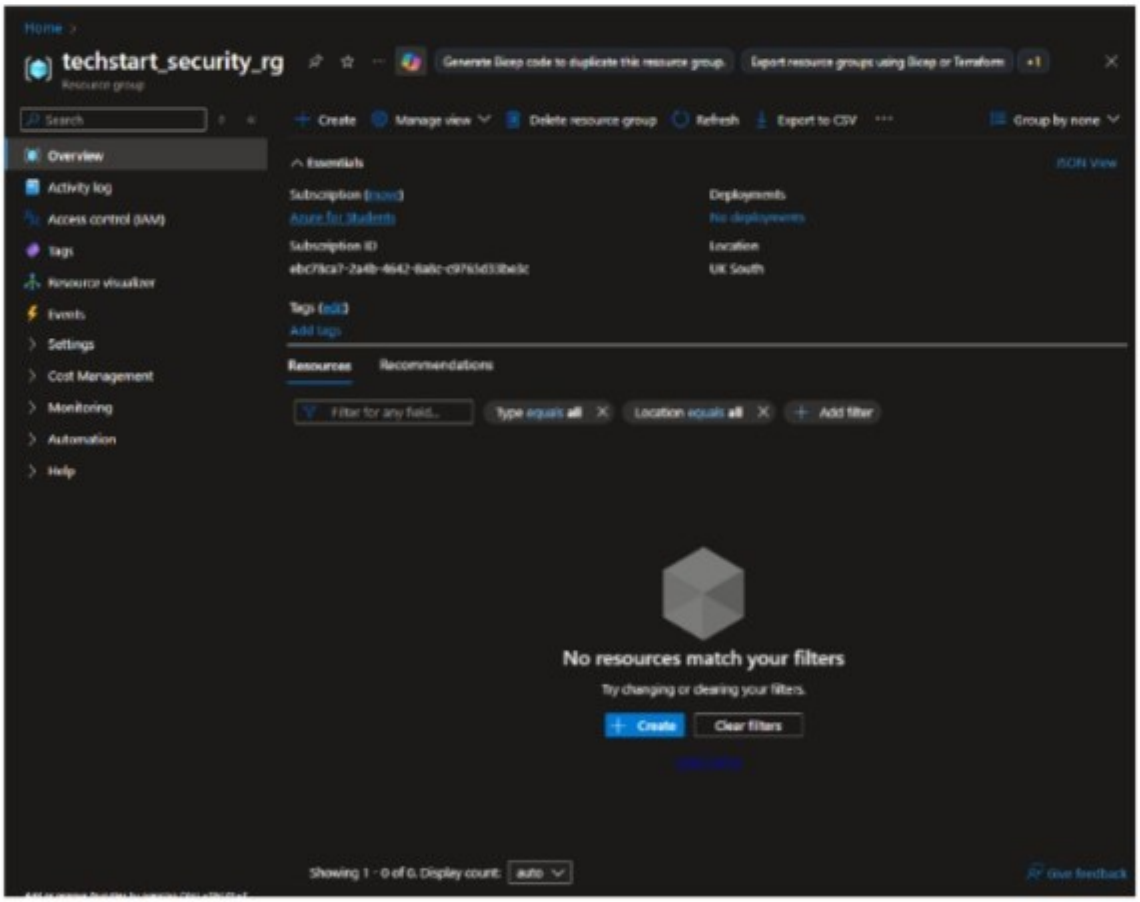


Figure 1: Resource Group

The virtual network, named 'techstart_vnet', with an address space of 10.0.0.0/16 has 2 subnets (Fig. 2). One subnet, named 'web_subnet', is for the virtual machine (Fig. 3). It has a

starting address 0f 10.0.1.0 with a /24 subnet mask. The other subnet, named 'db_subnet', is for the SQL database. It has a starting address of 10.0.2.0 with a /24 subnet mask. Using a /24 subnet mask inside of a larger /16 allows me to segment the network into smaller sub-networks which is perfect for containment and keeping the traffic of one subnet isolated from the other. I am also able to have up to 251 hosts per subnet, which is useful in larger organisations.

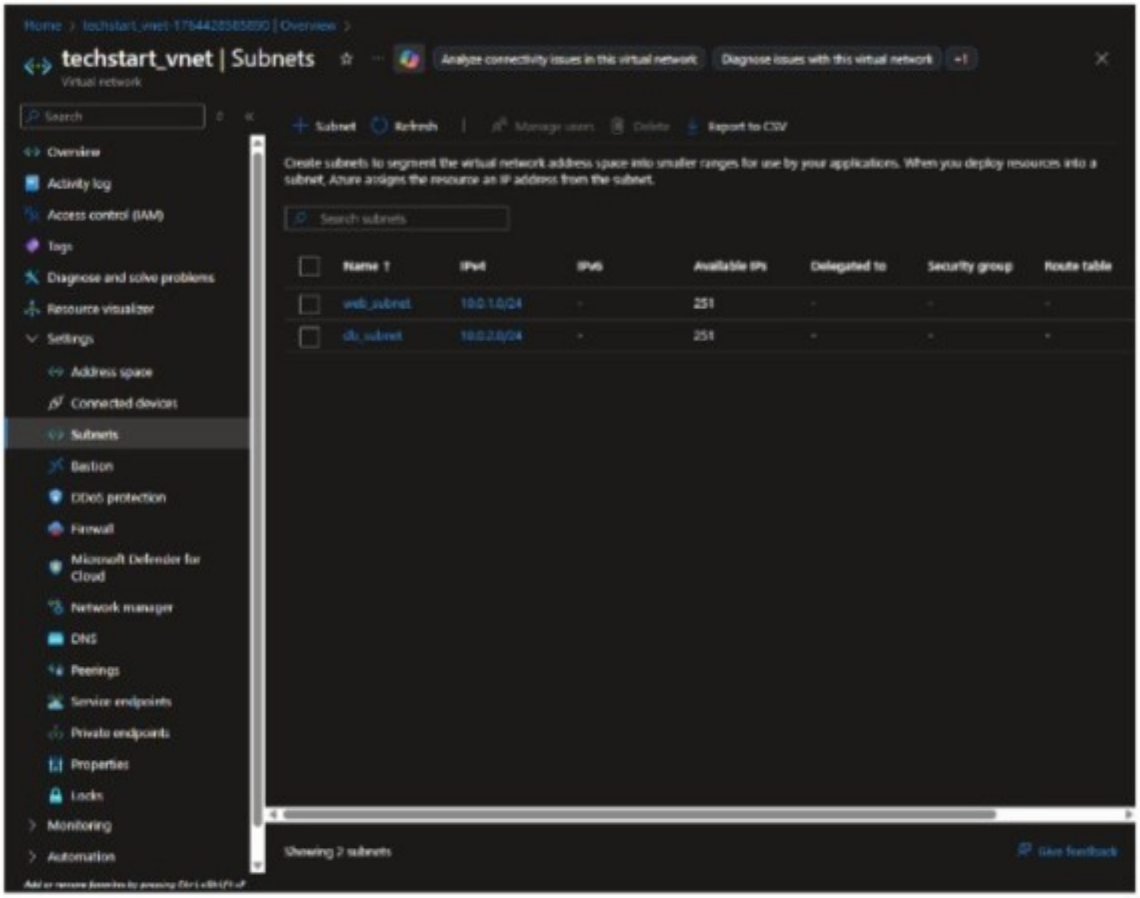


Figure 2: Subnets



Figure 3: Web Subnet Topology

The components in this project (as mentioned) are a firstly an Ubuntu 24.04 LTS virtual machine named 'techstart-webvm' that hosts the website. The virtual machine is connected to the 'web_subnet' subnet and has a public IP named 'techstart-webvm-ip' with IP '20.68.195.130' (Fig. 4). The NIC security group is set to 'advanced' since it will be configured in the 'Mandatory Security Requirements' section further below.

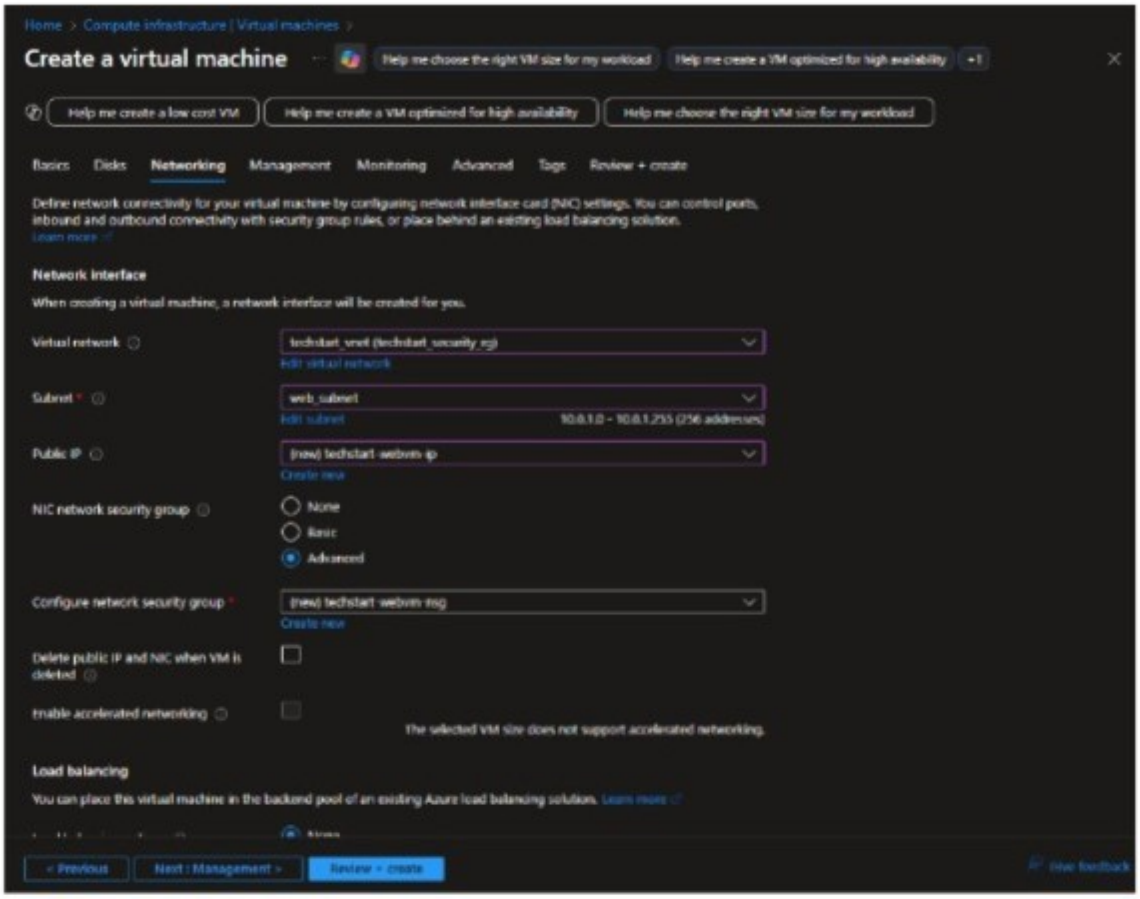


Figure 4: VM Network Settings

There is also an Azure SQL database named 'techstart_db' (Fig. 5), which stores all employee's information such as names, emails, and passwords. A new SQL database server was created named 'techstart-sql[redacted]' with UK South as its region. The authentication method for the server remains the default: SQL authentication. Concerning the database network settings, its connectivity method is set to public endpoint, and the allowed firewall rule is to add the current client IP address to the firewall rules, allowing connection from my current machine (Fig.6).

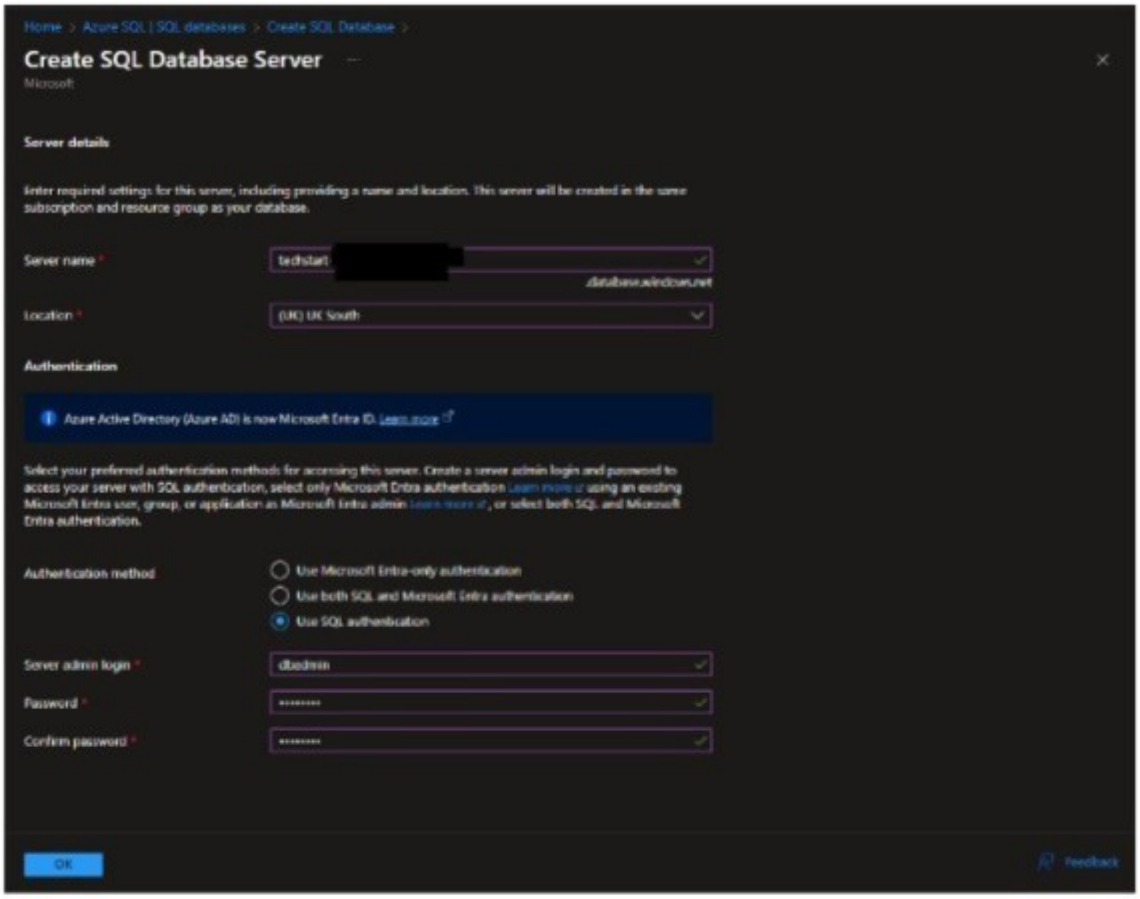


Figure 5: Creating SQL DB Server

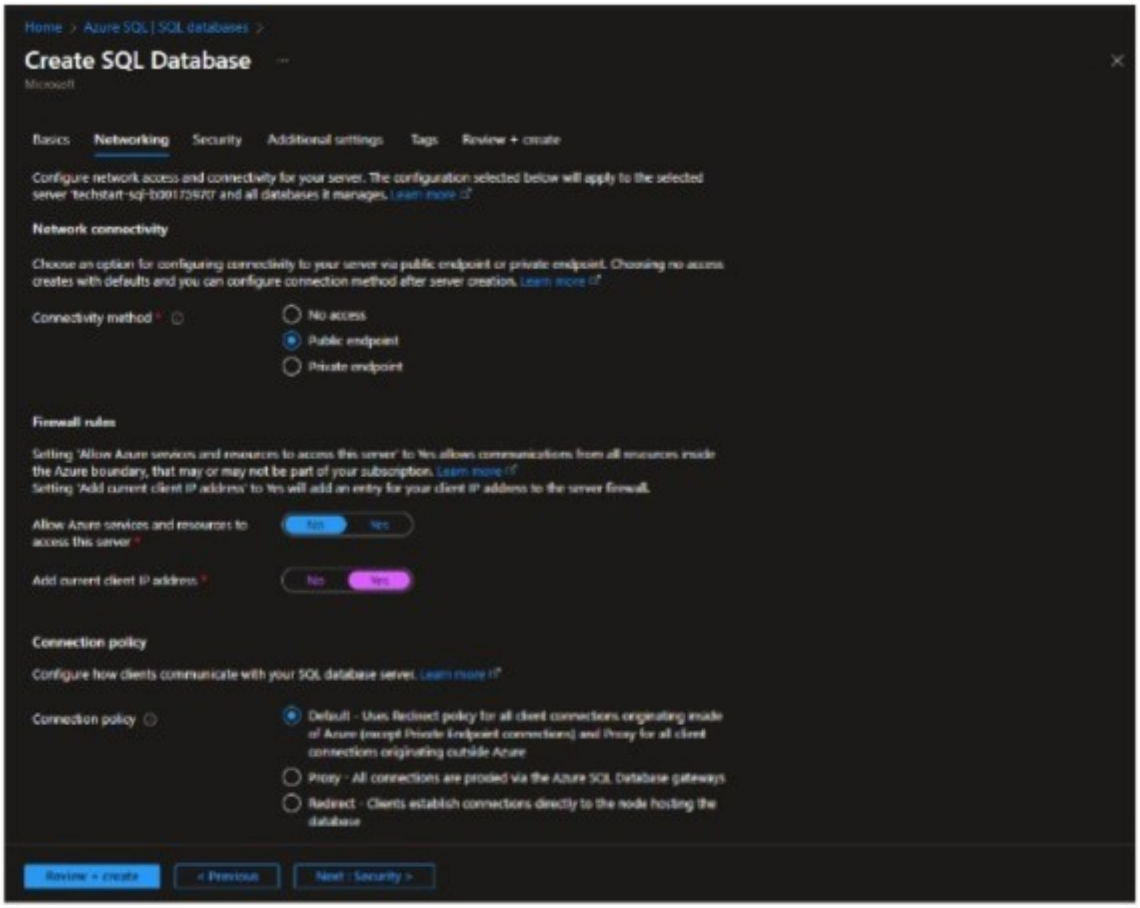


Figure 6: SQL DB Network Settings

And finally, there is Blob storage named 'techstorage' (Fig. 7) to store any kind of files needed. In this project, the only files I stored are in the 'profile-pictures' container (Fig. 8) which has the employees' profile pictures (Fig. 9) which can also be accessed by their respective links.

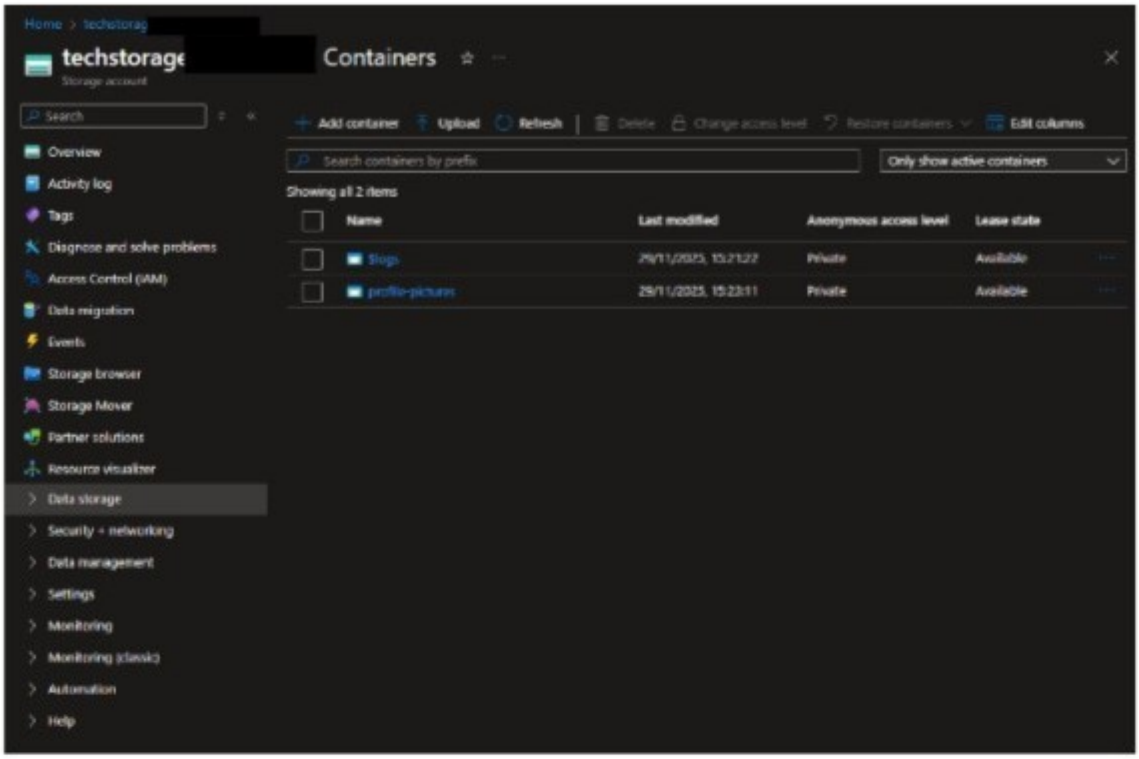


Figure 7: Containers



Figure 8: Creating Container

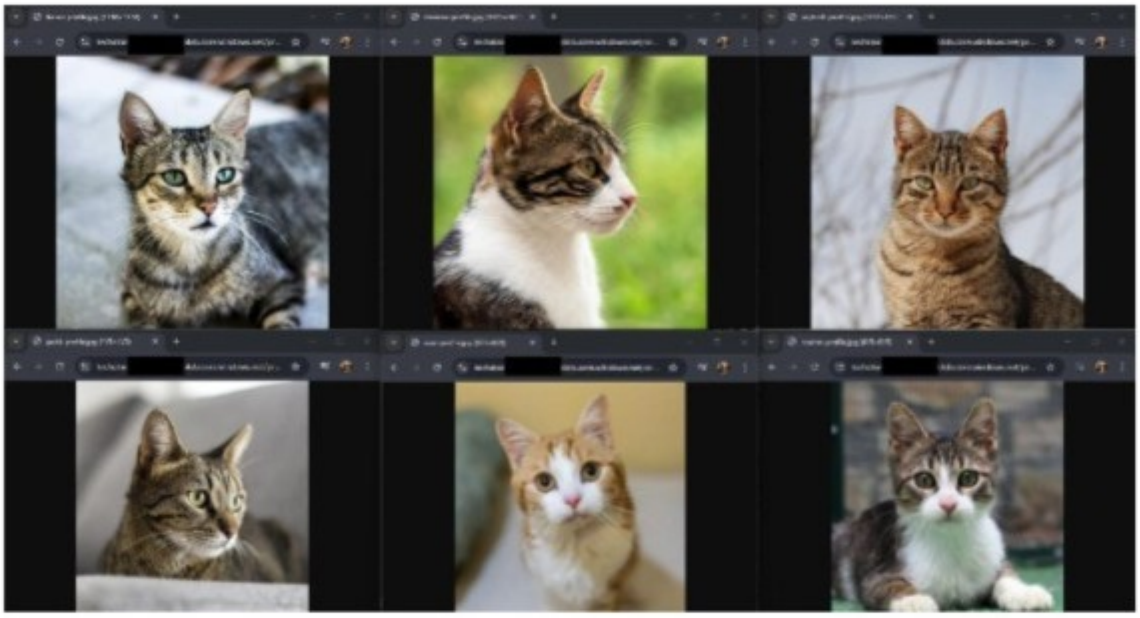


Figure 9: Profile Pictures

Mandatory Security Requirements

Requirement 1: Network Segmentation & Access Control

In the virtual machine, the Network Security Group (named 'techstart-webvm-nsg') blocks all traffic by default. It is the Host Network Perimeter. So, I allowed specific ports to be open: SSH (port 22) which is restricted only to the Admin's IP so that I may connect to the machine, and HTTPS (port443) which is allowed globally (Fig. 10). I also allowed HTTP (port 80) so that anyone that tries to connect to the website with HTTP, they will automatically be redirected to HTTPS (this is configured further below). All other ports are denied, stopping network discovery and unauthorised connections.

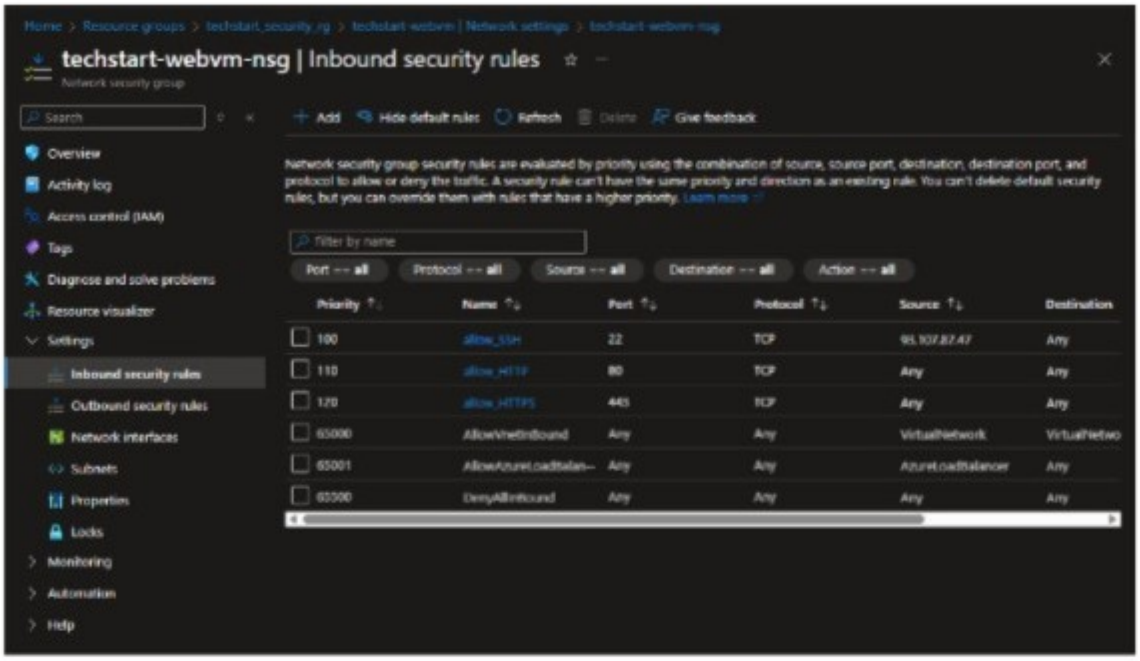


Figure 10: VM Inbound Security Rules

As for the SQL database, its firewall acts as a perimeter defence (specifically for the Platform as a Service) which ensures that the only traffic that reaches the database (on port 1433) originates from one trusted source only: the virtual machine's public IP address. So, since the database is isolated from the internet, I made it so that it only accepts connections from the virtual machine's public IP address (the trusted source). I did this by adding the firewall rule to the database directly from Azure (Fig. 11).

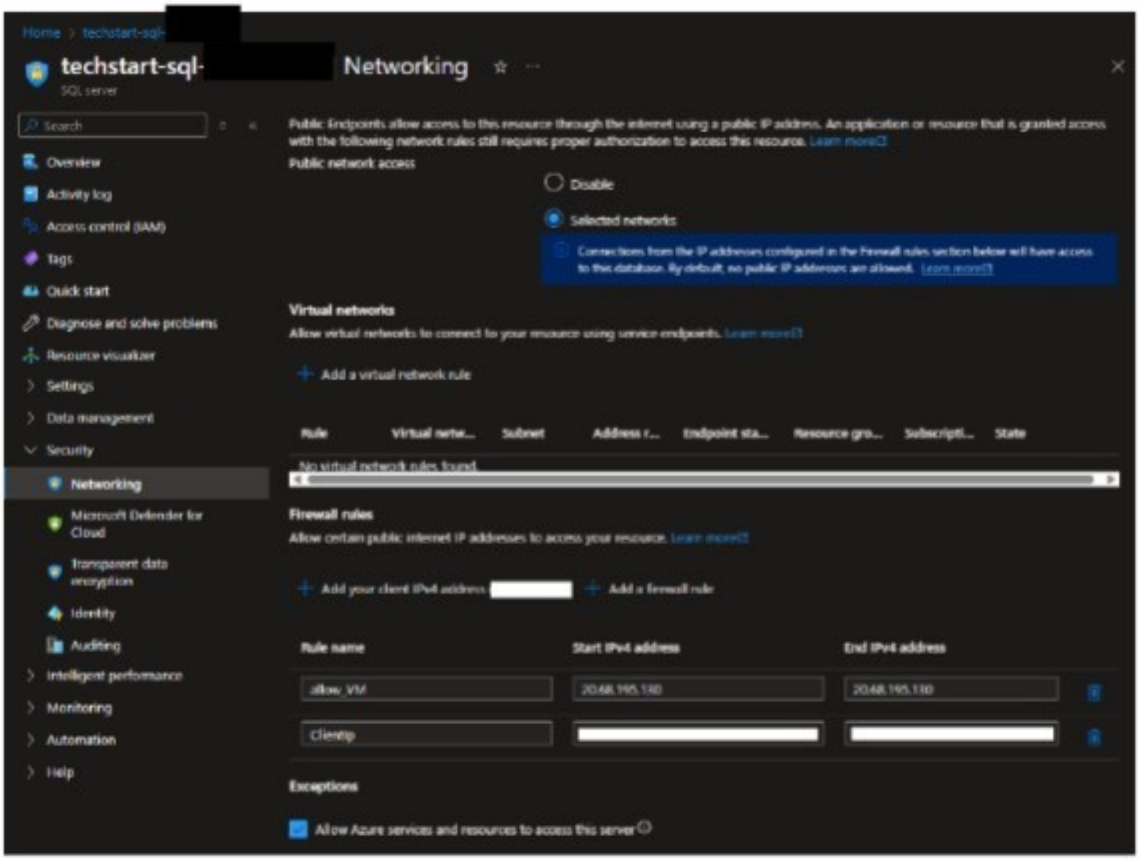


Figure 11: SQL Firewall Rules

Requirement 2: Strong Authentication & Access Management

The virtual machine can only be deployed using the downloaded 'techstart_key.pem' key file since I explicitly disabled password authentication in '/etc/ssh/sshd_config' to prevent brute-force dictionary attacks (Fig. 12). This means that only the user with access to that file may SSH to the virtual machine (which is me). I also disabled root login because many bots in brute-force attacks looking for open ssh ports usually try 'root@server'. Below are both the unsuccessful (password) login (Fig. 13) and the successful SSH login with the key (Fig. 14). The verbose debugging output of the SSH client connection attempt can be seen with the 'ssh -v' command (Fig. 15).

```
# Authentication:

#LoginGraceTime 2m
PermitRootLogin no
#StrictModes yes
#MaxAuthTries 6
#MaxSessions 10

PubkeyAuthentication yes

# Expect .ssh/authorized_keys2 to be disregarded by default in future.
#AuthorizedKeysFile     .ssh/authorized_keys .ssh/authorized_keys2

#AuthorizedPrincipalsFile none

#AuthorizedKeysCommand none
#AuthorizedKeysCommandUser nobody

# For this to work you will also need host keys in /etc/ssh/ssh_known_hosts
#HostbasedAuthentication no
# Change to yes if you don't trust ~/.ssh/known_hosts for
# HostbasedAuthentication
#IgnoreUserKnownHosts no
# Don't read the user's ~/.rhosts and ~/.shosts files
#IgnoreRhosts yes

# To disable tunneled clear text passwords, change to no here!
PasswordAuthentication no
#PermitEmptyPasswords no
```

Figure 12: Disable Password Auth & Root Login

```
[redacted]ssh -o PreferredAuthentications=password -o PubkeyAuthentication=no azureuser@20.68.195.130
[redacted].130: Permission denied (publickey).
```

Figure 13: Failed SSH Login

```
[redacted]@t
Welcome to Ubuntu 24.04.2 LTS (GNU/Linux 6.14.0-1018-azure x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:     https://landscape.canonical.com
 * Support:        https://ubuntu.com/pro

 System information as of Sat Nov 29 16:57:16 UTC 2025

  System load:  0.0               Processes:             113
  Usage of /:   5.4% of 28.02GB   Users logged in:       0
  Memory usage: 24%               IPv4 address for eth0: 10.0.1.4
  Swap usage:   0%

Expanded Security Maintenance for Applications is not enabled.

0 updates can be applied immediately.

Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status


The list of available updates is more than a week old.
To check for new updates run: sudo apt update


The programs included with the Ubuntu system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Ubuntu comes with ABSOLUTELY NO WARRANTY, to the extent permitted by
applicable law.

To run a command as administrator (user "root"), use "sudo <command>".
See "man sudo_root" for details.

azureuser@techstart-vm:~$
```

Figure 14: SSH Login



Figure 15: SSH Verbose

I installed Pam ('libpam-pwquality') (Fig.16) to configure password complexity at '/etc/pam.d/common-password' (Fig.17). I forced passwords to have a minimum length of 12 characters (minlen=12), with 3 characters in each new password that cannot be present in the old password (difok=3), and I allowed for up to 3 retries when entering the password (retry=3) (Fig. 18). This greatly increases security since the stronger the passwords, they are less susceptible they are to be retrieved by brute-force attacks.

```
azureuser@techstart-webvm:~$ sudo apt install libpam-pwquality
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following additional packages will be installed:
  cracklib-runtime libcrack2 libpwquality-common libpwquality1 wamerican
The following NEW packages will be installed:
  cracklib-runtime libcrack2 libpam-pwquality libpwquality-common libpwquality1 wamerican
0 upgraded, 6 newly installed, 0 to remove and 0 not upgraded.
Need to get 446 kB of archives.
After this operation, 1932 kB of additional disk space will be used.
Do you want to continue? [Y/n] Y
Get:1 http://azure.archive.ubuntu.com/ubuntu noble/main amd64 libcrack2 amd64 2.9.6-5.1build2 [29.0 k
B]
Get:2 http://azure.archive.ubuntu.com/ubuntu noble/main amd64 cracklib-runtime amd64 2.9.6-5.1build2
[147 kB]
Get:3 http://azure.archive.ubuntu.com/ubuntu noble/main amd64 libpwquality-common all 1.4.5-3build1 [
7748 B]
Get:4 http://azure.archive.ubuntu.com/ubuntu noble/main amd64 libpwquality1 amd64 1.4.5-3build1 [13.5
 kB]
Get:5 http://azure.archive.ubuntu.com/ubuntu noble/main amd64 libpam-pwquality amd64 1.4.5-3build1 [1
1.7 kB]
Get:6 http://azure.archive.ubuntu.com/ubuntu noble/main amd64 wamerican all 2020.12.07-2 [236 kB]
Fetched 446 kB in 0s (10.8 MB/s)
Preconfiguring packages ...
Selecting previously unselected package libcrack2:amd64.
(Reading database ... 68419 files and directories currently installed.)
Preparing to unpack .../0-libcrack2_2.9.6-5.1build2_amd64.deb ...
Unpacking libcrack2:amd64 (2.9.6-5.1build2) ...
Selecting previously unselected package cracklib-runtime.
Preparing to unpack .../1-cracklib-runtime_2.9.6-5.1build2_amd64.deb ...
Unpacking cracklib-runtime (2.9.6-5.1build2) ...
Selecting previously unselected package libpwquality-common.
Preparing to unpack .../2-libpwquality-common_1.4.5-3build1_all.deb ...
Unpacking libpwquality-common (1.4.5-3build1) ...
Selecting previously unselected package libpwquality1:amd64.
Preparing to unpack .../3-libpwquality1_1.4.5-3build1_amd64.deb ...
Unpacking libpwquality1:amd64 (1.4.5-3build1) ...
Selecting previously unselected package libpam-pwquality:amd64.
Preparing to unpack .../4-libpam-pwquality_1.4.5-3build1_amd64.deb ...
Unpacking libpam-pwquality:amd64 (1.4.5-3build1) ...
Selecting previously unselected package wamerican.
Preparing to unpack .../5-wamerican_2020.12.07-2_all.deb ...
Unpacking wamerican (2020.12.07-2) ...
Setting up libpwquality-common (1.4.5-3build1) ...
Setting up wamerican (2020.12.07-2) ...
Setting up libcrack2:amd64 (2.9.6-5.1build2) ...
Setting up cracklib-runtime (2.9.6-5.1build2) ...
Setting up libpwquality1:amd64 (1.4.5-3build1) ...
Setting up libpam-pwquality:amd64 (1.4.5-3build1) ...
Processing triggers for libc-bin (2.39-0ubuntu8.6) ...
Processing triggers for man-db (2.12.0-4build2) ...
Scanning processes...
Scanning linux images...

Running kernel seems to be up-to-date.
```

Figure 16: Installing Pam

```
#
# /etc/pam.d/common-password - password-related modules common to all services
#
# This file is included from other service-specific PAM config files,
# and should contain a list of modules that define the services to be
# used to change user passwords.  The default is pam_unix.

# Explanation of pam_unix options:
# The "yescrypt" option enables
#hashed passwords using the yescrypt algorithm, introduced in Debian
#11.  Without this option, the default is Unix crypt.  Prior releases
#used the option "sha512"; if a shadow password hash will be shared
#between Debian 11 and older releases replace "yescrypt" with "sha512"
#for compatibility .  The "obscure" option replaces the old
# `OBSCURE_CHECKS_ENAB' option in login.defs.  See the pam_unix manpage
#for other options.

# As of pam 1.0.1-6, this file is managed by pam-auth-update by default.
# To take advantage of this, it is recommended that you configure any
# local modules either before or after the default block, and use
# pam-auth-update to manage selection of other modules.  See
# pam-auth-update(8) for details.

# here are the per-package modules (the "Primary" block)
password        requisite                       pam_pwquality.so retry=3 minlen=12 difok=3
password        [success=1 default=ignore]      pam_unix.so obscure use_authtok try_first_pass yescr
# here's the fallback if no module succeeds
password        requisite                       pam_deny.so
# prime the stack with a positive return value if there isn't one already;
# this avoids us returning an error just because nothing sets a success code
# since the modules above will each just jump around
password        required                        pam_permit.so
# and here are more per-package modules (the "Additional" block)
# end of pam-auth-update config
```

Figure 17: Configuring Pam



Figure 18: Pam Complexity Rules

Requirement 3: Encryption in Transit and at Rest

Azure SQL transparent data encryption is enabled by default for the At Rest state. It essentially protects all data on the disk. And Blob storage is also encrypted by default with keys managed by Microsoft.

I chose to use HTTPS instead of HTTP because it adds a layer of security called Transport Layer Security which encrypts all data exchanged between the browser and the server, ensuring confidentiality. For the HTTPS I generated a self-signed certificate (OpenSSL) and used the Apache configuration to enforce HTTPS (on port 443) (Fig. 19-20). And SSL is active because the Python app enforces SSL for database connections ('Encrypt=yes' on line 19) (Fig. 21).

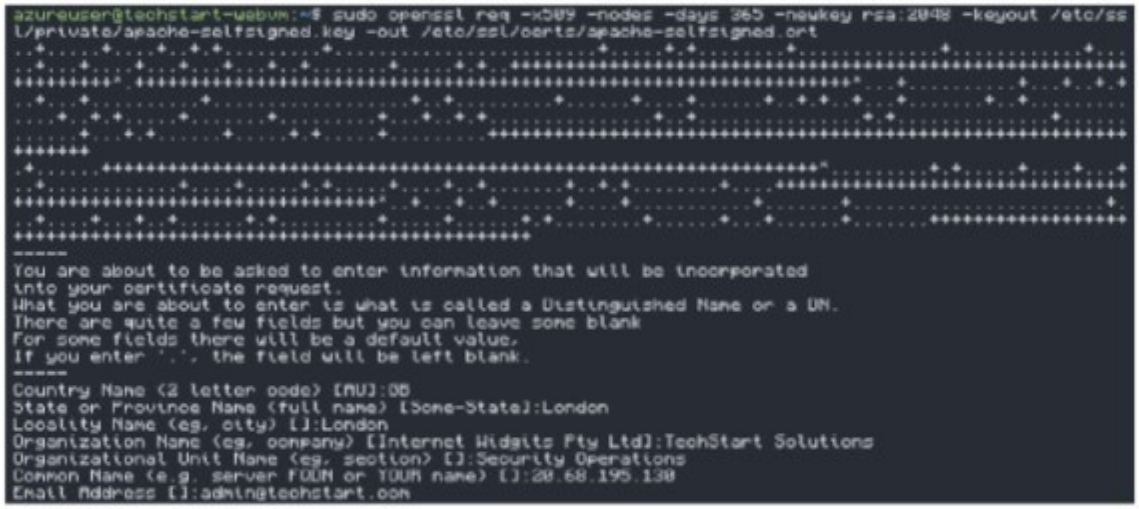


Figure 19: HTTPS Certificate (1)



Figure 20: HTTPS Certificate (2)

app.py

```python
from flask import Flask, render_template, request, redirect, url_for, session
import pyodbc
import os
from werkzeug.security import check_password_hash

app = Flask(__name__)
app.secret_key = os.environ.get('SECRET_KEY', 'default-dev-key')

BLOB_URL = "https://techstorageb00175970.blob.core.windows.net/profile-pictures/"

# Database Connection Function
def get_db_connection():
    conn_str = (
        f"DRIVER={{ODBC Driver 18 for SQL Server}};"
        f"SERVER={os.environ.get('SQL_SERVER')};"
        f"DATABASE={os.environ.get('SQL_DATABASE')};"
        f"UID={os.environ.get('SQL_USERNAME')};"
        f"PWD={os.environ.get('SQL_PASSWORD')};"
        "Encrypt=yes;TrustServerCertificate=no;Connection Timeout=30;"
    )
    return pyodbc.connect(conn_str)

@app.route('/')
def home():
    if 'user' in session:
        return render_template('dashboard.html', user=session['user'], role=session['role'])
    return render_template('login.html')

@app.route('/login', methods=['POST'])
def login():
    email = request.form['email']
    password = request.form['password']

    try:
        conn = get_db_connection()
```

Figure 21: Enforcing SSL

Requirement 4: System Hardening & Monitoring

I configured the host-based firewall with Uncomplicated Firewall to deny all incoming data by default (for system hardening) and allowed all outgoing data by default (Fig. 22-23). The Host Firewall provides a local internal security boundary protecting the running services even if the Azure Network Resource Group was misconfigured. This represents the 'Defence in Depth' principle.

```
azureuser@techstart-webvm:~$ sudo ufw default deny incoming
Default incoming policy changed to 'deny'
(be sure to update your rules accordingly)
azureuser@techstart-webvm:~$ sudo ufw default allow outgoing
Default outgoing policy changed to 'allow'
(be sure to update your rules accordingly)
azureuser@techstart-webvm:~$ sudo ufw allow ssh
Rules updated
Rules updated (v6)
azureuser@techstart-webvm:~$ sudo ufw allow 80/tcp
Rules updated
Rules updated (v6)
azureuser@techstart-webvm:~$ sudo ufw allow 443/tcp
Rules updated
Rules updated (v6)
azureuser@techstart-webvm:~$ sudo ufw enable
Command may disrupt existing ssh connections. Proceed with operation (y|n)? y
Firewall is active and enabled on system startup
```

Figure 22: Configuring UFW

```
azureuser@techstart-webvm:~$ sudo ufw status verbose
Status: active
Logging: on (low)
Default: deny (incoming), allow (outgoing), disabled (routed)
New profiles: skip

To                         Action      From
--                         ------      ----
22/tcp                     ALLOW IN    Anywhere
80/tcp                     ALLOW IN    Anywhere
443/tcp                    ALLOW IN    Anywhere
443                        ALLOW IN    Anywhere
22/tcp (v6)                ALLOW IN    Anywhere (v6)
80/tcp (v6)                ALLOW IN    Anywhere (v6)
443/tcp (v6)               ALLOW IN    Anywhere (v6)
443 (v6)                   ALLOW IN    Anywhere (v6)
```

Figure 23: UFW Status

I also installed and configured unattended-upgrades to ensure security patches are applied automatically without admin intervention (further enhancing system hardening) (Fig. 24-26).

```
azureuser@techstart-webvm:~$ sudo apt install unattended-upgrades
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
unattended-upgrades is already the newest version (2.9.1+nmu4ubuntu1).
unattended-upgrades set to manually installed.
0 upgraded, 0 newly installed, 0 to remove and 0 not upgraded.
azureuser@techstart-webvm:~$ sudo dpkg-reconfigure --priority=low unattended-upgrades
azureuser@techstart-webvm:~$
```

Figure 24: Installing Unattended Upgrades



Figure 25: Configuring Unattended Upgrades



Figure 26: Unattended Upgrades Status

BASH Scripts

I firstly created a security hardening BASH script that executes 3 simple tasks: updating and upgrading packages, configuring the UFW rules mentioned above (and enabling UFW), and checking for open ports (it provides a direct view of the potential attack surface of the system) (Fig. 27-29).

```
#!/bin/bash

echo "Starting Security Hardening..."

# Update package list
echo -e "\n Updating packages..." # The '-e' and '\n' skip a line
sudo apt update && sudo apt upgrade -y  # This updates all packages

# Configure UFW
echo -e "\n Configuring UFW..."
sudo ufw allow ssh      # This allows SSH (port 22)
sudo ufw allow http     # This allows HTTP (port 80)
sudo ufw allow https    # This allows HTTPS (port 443)
sudo ufw --force enable # This forces the script to enable UFW

# Check for open ports
echo -e "\n Current Open Ports:"
sudo ss -tulpn          # This checks for all currently open ports

echo -e "\n Hardening Complete"
```

Figure 27: harden.sh BASH Script

```
  GNU nano 7.2    harden.sh
#!/bin/bash

echo "Starting Security Hardening..."

# Update package list
echo -e "\n Updating packages..."
sudo apt update && sudo apt upgrade -y

# Configure UFW
echo -e "\n Configuring UFW..."
sudo ufw allow ssh
sudo ufw allow http
sudo ufw allow https
sudo ufw --force enable

# Check for open ports
echo -e "\n Current Open Ports:"
sudo ss -tulpn

echo -e "\n Hardening Complete."
```

Figure 28: Creating Hardening Script

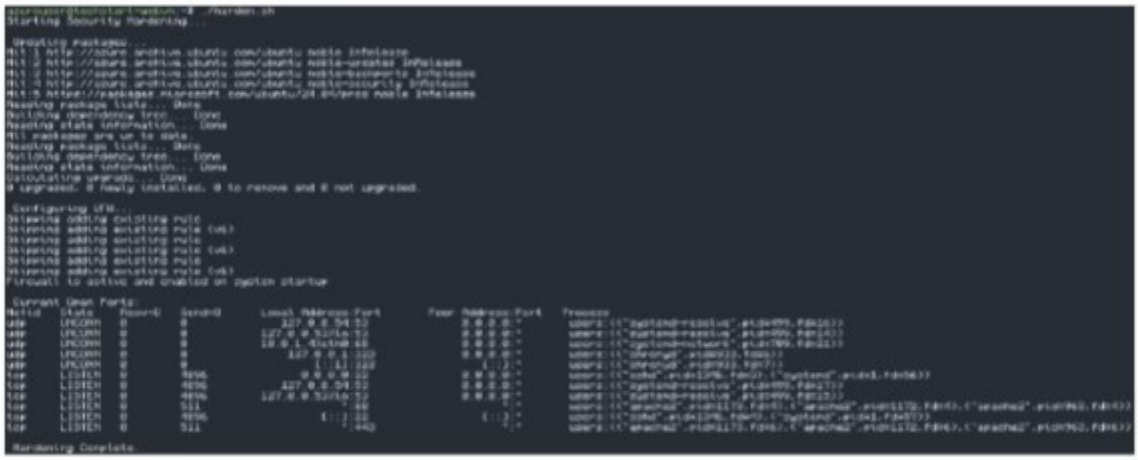


Figure 29: Running Hardening Script

I also created a security monitoring BASH script that executes the 4 following tasks: checking the firewall's status with UFW to ensure it is active (confirming the host perimeter is active), checking that the SSH brute-force protection (Fail2Ban will be explained more in the Additional Security Measures section below) is active, checking the last 5 failed login attempts (providing real-time Audit Log for SSH which demonstrates proactive threat detection), and finally checking the top 10 processes that are running as root (serving as a rapid check for malware or unauthorised background services that may have escalated privileges) (Fig. 30-32).

```
#!/bin/bash

echo "Starting Security Monitoring..."

# Check Firewall Status (UFW)
echo -e "\n Host Firewall Status (UFW):"
if sudo ufw status | grep -q "Status: active"; then # This searches for
the characters 'Status: active' in the UFW status output using grep (the
'-q' is quite mode so no output is printed)
    echo "UFW is active and running."
    sudo ufw status | grep -v "Status: active" # This finds all results in
the output that do not include the selected characters (using '-v')
else
    echo "ALERT: UFW is INACTIVE."
fi

# Check SSH Brute-Force Protection (Fail2Ban)
echo -e "\n Brute-Force Protection Status (Fail2Ban):"
if sudo systemctl status fail2ban | grep -q "active (running)"; then #
This also searches for characters in the fail2ban status output
    echo "Fail2Ban service is running."
    sudo fail2ban-client status sshd | grep "Currently banned"
else
    echo "ALERT: Fail2Ban is NOT running."
fi

# Check Recent Failed Login Attempts
echo -e "\n Recent Failed Login Attemps:"
sudo grep "Failed password for" /var/log/auth.log | tail -n 5 # This
displays the last 5 failed login attempts recorded ('tail' represents the
last lines and '-n 5' is the number of lines wanted)
if [ $? -ne 0 ]; then
    echo "No failed login attempts found."
fi

# Check Root Processes
echo -e "\n Processes Running as Root:"
ps -eo user,pid,pcpu,pmem,command --sort=-pcpu | head -n 10 # The '--
sort=pcpu' is to sort the outputted processes by their CPU utilisation
percentage (and 'head' represents the first lines and '-n 10' the number
of lines wanted)

echo "Monitoring complete."
```

Figure 30: monitor.sh BASH Script

```
  GNU nano 7.2                          monitor.sh
#!/bin/bash

echo "Starting Security Monitoring..."

# Check Firewall Status (UFW)
echo -e "\n Host Firewall Status (UFW):"
if sudo ufw status | grep -q "Status: active"; then
    echo "UFW is active and running."
    sudo ufw status | grep -v "Status: active"
else
    echo "ALERT: UFW is INACTIVE."
fi

# Check SSH Brute-Force Protection (Fail2Ban)
echo -e "\n Brute-Force Protection Status (Fail2Ban):"
if sudo systemctl status fail2ban | grep -q "active (running)"; then
    echo "Fail2Ban service is running."
    sudo fail2ban-client status sshd | grep "Currently banned"
else
    echo "ALERT: Fail2Ban is NOT running."
fi

# Check Recent Failed Login Attempts
echo -e "\n Recent Failed Login Attemps:"
sudo grep "Failed password for" /var/log/auth.log | tail -n 5
if [ $? -ne 0 ]; then
    echo "No failed login attempts found."
fi

# Check Root Processes
echo -e "\n Processes Running as Root:"
ps -eo user,pid,ppu,pmem,command --sort=-ppu | head -n 10

echo "Monitoring complete."
```

Figure 31: Creating Monitoring Script

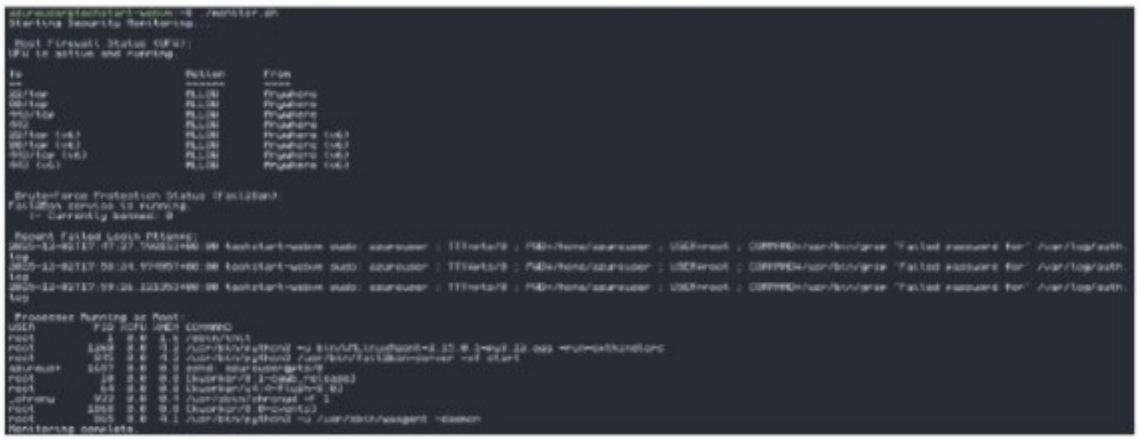


Figure 32: Running Monitoring Script

Requirement 5: Logging, Backup & Audit Trail

I firstly used logrotate to manage log file sizes and retention periods to prevent disk exhaustion. I used the default settings which is configured to automatically compress, archive, and rotate log files on a weekly basis, retaining four weeks of history (Fig. 33). This ensures that the logs remain manageable in size, preventing disk space exhaustion while complying with short-term audit requirements.

```
azureuser@techstart-webvm:~$ cat /etc/logrotate.conf
# see "man logrotate" for details

# global options do not affect preceding include directives

# rotate log files weekly
weekly

# use the adm group by default, since this is the owning group
# of /var/log/.
su root adm

# keep 4 weeks worth of backlogs
rotate 4

# create new (empty) log files after rotating old ones
create

# use date as a suffix of the rotated file
#dateext

# uncomment this if you want your log files compressed
#compress

# packages drop log rotation information into this directory
include /etc/logrotate.d

# system-specific logs may also be configured here.
```

Figure 33: Log Retention

To ensure the recoverability of the whole system, I also created a backup for the virtual machine which in Azure with Azure's Backup service (Fig. 34). This provides the ability to perform a full disaster recovery of the virtual machine, restoring the entire operating system, Apache configuration, and Flask application code to a working state in the event of hardware failure, catastrophic OS corruption, or a successful ransomware attack.

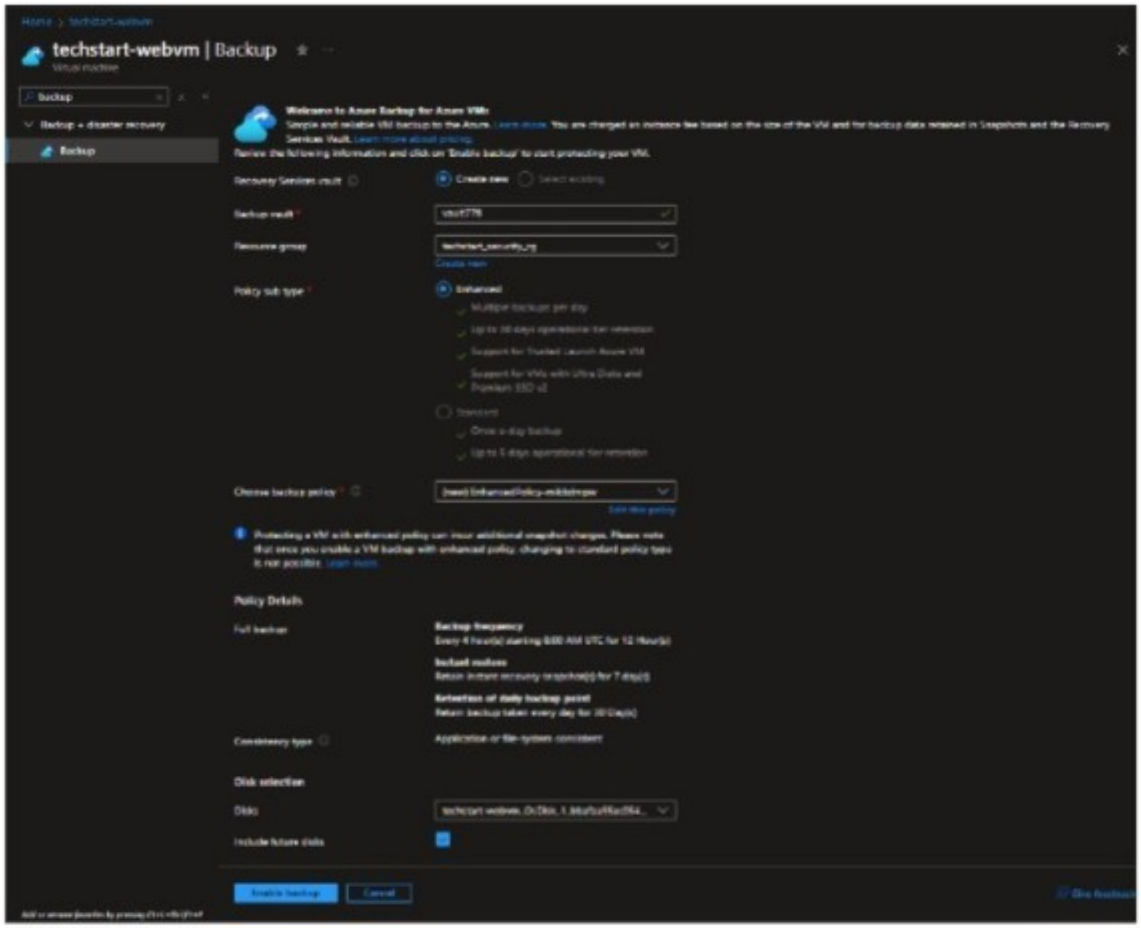


Figure 34: Azure Backup

Additional Security Measures

For the first additional security measure, I hashed all passwords for the staff (aligning with system hardening). I installed python's bcrypt library (which was already installed) (Fig. 35) which I used in my 'login' function in my 'app.py' script (Line 32-34) (Fig. 36). The SQL database was adapted for this (Fig. 49).

```
azureuser@techstart-webvm:~$ sudo pip3 install pyodbc bcrypt --break-system-packages
Requirement already satisfied: pyodbc in /usr/local/lib/python3.12/dist-packages (5.3.0)
Requirement already satisfied: bcrypt in /usr/lib/python3/dist-packages (3.2.2)
WARNING: Running pip as the 'root' user can result in broken permissions and conflicting
behaviour with the system package manager. It is recommended to use a virtual environment
 instead: https://pip.pypa.io/warnings/venv
```

Figure 35: Installing bcrypt



```python
@app.route('/login', methods=['POST'])
def login():
    email = request.form['email']
    password = request.form['password']
    try:
        conn = get_db_connection()
        cursor = conn.cursor()
        cursor.execute("SELECT full_name, role, password_hash FROM staff_directory WHERE email = ?", email)
        row = cursor.fetchone()
        conn.close()
        if row:
            input_password_bytes = password.encode('utf-8')
            stored_hash_bytes = row.password_hash.encode('utf-8')
            if checkpw(input_password_bytes, stored_hash_bytes):
                session['user'] = row.full_name
                session['role'] = row.role
                name_prefix = email.split('@')[0].split('.')[0]
                initials = ''.join([n[0].lower() for n in row.full_name.split() if n]).replace("'", "")
                name_mapping = {
                    'Liam O\'Connor': 'liamo',
                    'Sophie Byrne': 'sophieb',
                    'Noah Murphy': 'noahm',
                    'Emma Walsh': 'emmaw',
                    'Jack Kelly': 'jackk',
                    'Ava Ryan': 'avar',
                }
                user_filename_prefix = name_mapping.get(row.full_name, 'default')
                profile_pic_url = f"{BLOB_URL}{user_filename_prefix}-profile.jpg"
                session['profile_pic'] = profile_pic_url
                return redirect(url_for('home'))
        return "Invalid Credentials", 401
    except Exception as e:
        return f"Database Error: {e}", 500
```

Figure 36: Login Function

I installed the Linux Audit Daemon (Auditd) to monitor file integrity, such as changes to '/etc/passwd' (Fig. 37-39). This helps detect unauthorized access, system tampering, and potential security breaches.



Figure 37: Installing Auditd



Figure 38: Enabling Auditd

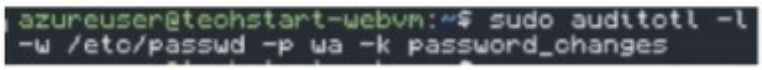


Figure 39: Auditd Results

I also installed Fail2Ban to ban IPs that fail SSH login repeatedly, preventing brute-force attacks (Fig. 40-41). Fail2Ban involves regex monitoring of '/var/log/auth.log' and dynamic Iptables/UFW blocking which is why it is superior to simply having strong passwords. It also prevents resource exhaustion attacks like denial-of-service.



Figure 40: Installing Fail2Ban



Figure 41: Fail2Ban Status

Access Control Matrix

All the employees are only application-level users authenticated with the Python web app, whereas the system administrator (me) is an infrastructure-level user authenticated with Azure Entra ID and SSH keys. I attempted to follow the Principle of Least Privilege (PoLP) as best as I could. Essentially, all employees, whether they are developers, QA testers or project managers will have no access on an infrastructure-level (Azure Layer). They only have access strictly to what they need.

What all staff can do is login and view their respective dashboards as well as viewing information that is publicly available such as Blob Access (profile pictures can be accessed with a simple link (Fig. 8)).

I then set each team to have specific access to specific resources: The developers can read and write the virtual machine and the database only to support infrastructure-as-code deployments (I am deliberately violating PoLP in specific areas to facilitate rapid DevOps deployment), QA Testers are responsible for direct management of static assets via Blob Storage so they have only read and write for Blob storage, and finally project managers can view virtually everything (Table 1).

No staff can directly manipulate the virtual machine hosting the website (to turn it on/off) nor login with SSH to connect to it (since they do not have the key file 'techstart_key.pem' which only I (the admin) have). The information displayed on their dashboard represents also what they can do, it explains what each of their job is.

In addition, the dashboard page ('dashboard.html') uses conditional logic ('{% if role == '...' %}') to only show the necessary components, which restricts functionality based on the user's role.

Table 1: TechStart Solutions Access Control Matrix

Users	Team	Azure Role	OS Access	VM Access	DB Access	Blob Access	App Dashboard View
Liam O'Connor Sophie Byrne	Developers	TechStart_ Developer	Denied	Read & Write	Read & Write	Read (Public)	VM/DB Configuration Access
Noah Murphy Emma Walsh	QA Testers	TechStart_QA	Denied	Denied	Denied	Read & Write	Storage Management Console
Jack Kelly Ava Ryan	Project Managers	TechStart_PM	Denied	Read Only	Read Only	Read (Public)	High-Level Status Oversight
azureuser	SysAdmin	Owner / Contributor	Full (Key-based)	Full Control	Full Admin	Full Control	System Config

Finally I created the Azure roles for each staff member (Fig.42-45). However, due to administrative restrictions by the university, the creation of new Azure accounts to assign the custom roles to is not possible.

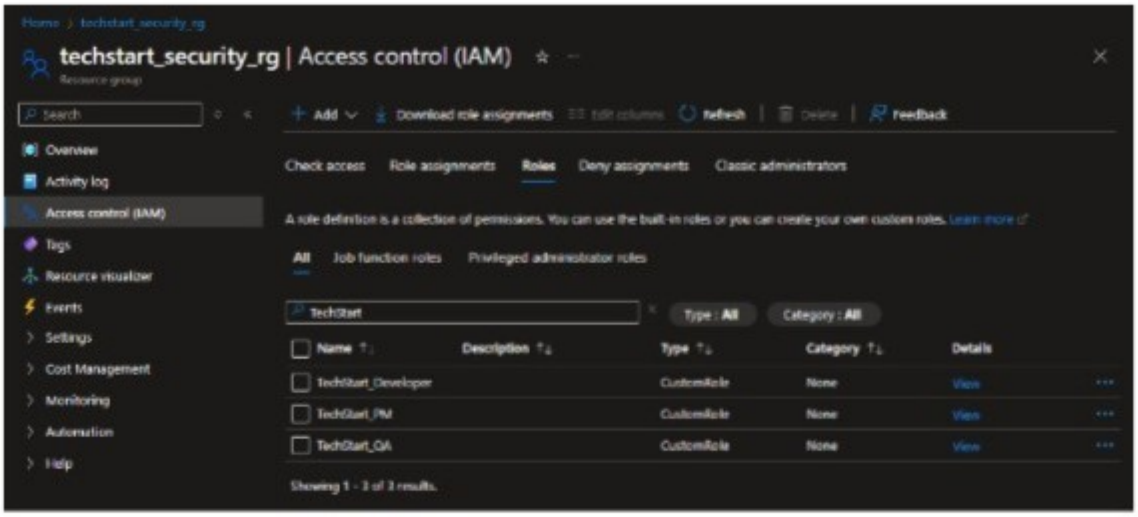


Figure 42: Team Roles



Figure 43: Developer Permissions

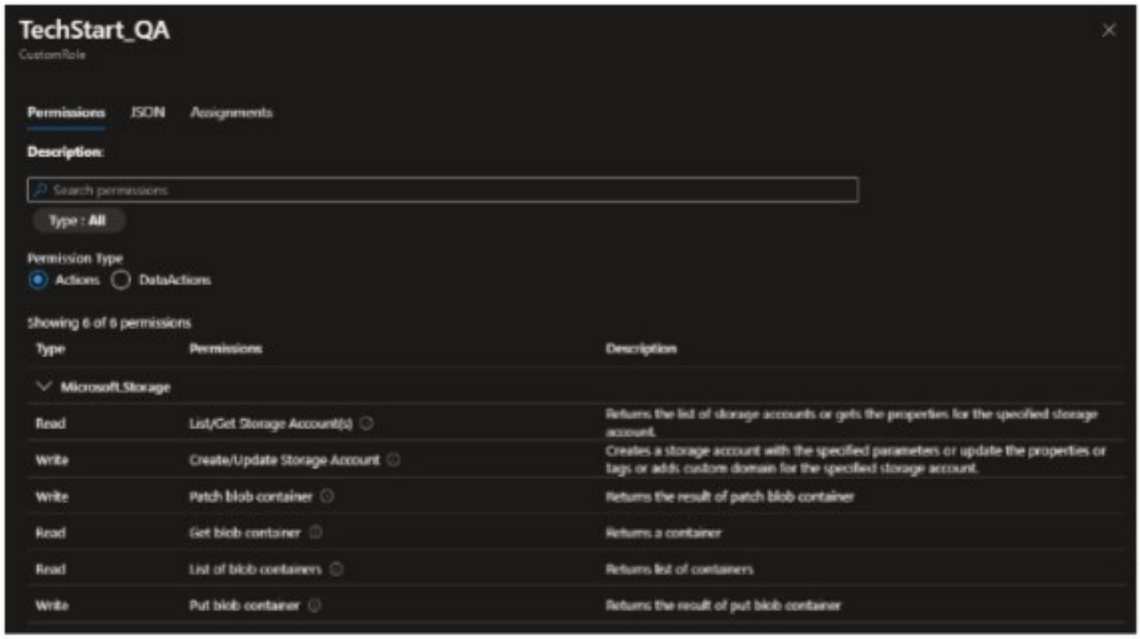


Figure 44: QA Tester Permissions



Figure 45: Project Manager Permissions

App Deployment

To finally get to the deployment of the app, I first installed all dependencies for the system: Python Flask library for the 'app.py' file, launching the app (Fig. 46), and the OBDC Driver 18 for the SQL Server to allow the website's code to connect to the server (Fig. 47-50).

```
azureuser@techstart-webvm:~$ sudo apt install python3-pip python3-flask apache2 libapache2-mod-wsgi-p
y3 unixodbc-dev -y
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following additional packages will be installed:
  apache2-bin apache2-data apache2-utils binutils binutils-common binutils-x86-64-linux-gnu
  build-essential bzip2 cpp cpp-13 cpp-13-x86-64-linux-gnu cpp-x86-64-linux-gnu dpkg-dev fakeroot
  fontconfig-config fonts-dejavu-core fonts-dejavu-mono g++ g++-13 g++-13-x86-64-linux-gnu
  g++-x86-64-linux-gnu gcc gcc-13 gcc-13-base gcc-13-x86-64-linux-gnu gcc-x86-64-linux-gnu
  javascript-common libalgorithm-diff-perl libalgorithm-diff-xs-perl libalgorithm-merge-perl
  libaom3 libapr1t64 libaprutil1-dbd-sqlite3 libaprutil1-ldap libaprutil1t64 libasan8 libatomic1
  libbinutils libc-dev-bin libc-devtools libc6-dev libcc1-0 liberypt-dev libdf-nobfd0 libdf0
  libde265-0 libdeflate0 libdpkg-perl libexpat1-dev libfakeroot libfile-fontlock-perl
  libfontconfig1 libgcc-13-dev libgd3 libgomp1 libgprofng0 libheif-plugin-aomdec
  libheif-plugin-aomenc libheif-plugin-libde265 libheif1 libhwasan0 libisl23 libitm1 libjbig0
  libjpeg-turbo8 libjpeg8 libjs-jquery libjs-sphinxdoc libjs-underscore liblerc4 liblsan0 libltdl7
  liblua5.4-0 libmpc3 libodbc2 libodbcinst2 libpython3-dev libpython3.12-dev
  libquadmath0 libsframe1 libsharpyuv0 libstdc++-13-dev libtiff6 libtsan2 libubsan1 libwebp7
  libxpm4 linux-libc-dev lto-disabled-list make manpages-dev python3-asgiref python3-dev
  python3-itsdangerous python3-simplejson python3-werkzeug python3-wheel python3.12-dev
  rpcsvc-proto ssl-cert unixodbc-common zlib1g-dev
Suggested packages:
  apache2-doc apache2-suexec-pristine | apache2-suexec-custom www-browser binutils-doc gprofng-gui
  bzip2-doc cpp-doc gcc-13-locales cpp-13-doc debian-keyring g++-multilib g++-13-multilib
  gcc-13-doc gcc-multilib autoconf automake libtool flex bison gdb gcc-doc gcc-13-multilib
  gdb-x86-64-linux-gnu glibc-doc bzr libgd-tools libheif-plugin-x265 libheif-plugin-ffmpegdec
  libheif-plugin-jpegdec libheif-plugin-jpegenc libheif-plugin-j2kdec libheif-plugin-j2kenc
  libheif-plugin-rav1e libheif-plugin-svtenc odbc-postgresql tdsodbc libstdc++-13-doc make-doc
  python3-dotenv python-flask-doc ipython3 python-werkzeug-doc python3-lxml python3-watchdog
The following NEW packages will be installed:
```

Figure 46: Installing Dependencies (1)

```
azureuser@techstart-webvm:~$ curl https://packages.microsoft.com/config/ubuntu/$(lsb_release -rs)/pro
d.list | sudo tee /etc/apt/sources.list.d/mssql-release.list
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed
100   138  100   138    0     0   2240      0 --:--:-- --:--:-- --:--:--  2262
deb [arch=amd64,arm64,armhf signed-by=/usr/share/keyrings/microsoft-prod.gpg] https://packages.micros
oft.com/ubuntu/24.04/prod noble main
azureuser@techstart-webvm:~$ curl https://packages.microsoft.com/keys/microsoft.asc | sudo tee /etc/a
pt/trusted.gpg.d/microsoft.asc
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed
100   975  100   975    0     0  16986      0 --:--:-- --:--:-- --:--:-- 16810
-----BEGIN PGP PUBLIC KEY BLOCK-----
Version: BSN Pgp v1.1.0.0

mQENBFYxWIwBCADMNoZhZlJxDNOWzqV+1OO1xiOeoowW9hssOMNvd+buXCOI5ZJwT
LAZqlolLF7pqdoZUtt9b3o7yBT2MalDp9LiuWEokyuO6VVA1T72MFf5Eu6x6OLV
7aVUsCzUMF+qb7DC9fFuFLEdxNOETaFJzr07oCn3V4JDxMqNU4T6OJpvMazOl3ag
OetzFXxMiLjMtUUttMOZnMhtlzkmwlrUtvbffFD+fEoMJ1+uldf0zZN8/oKMMLe2J
M6J2kvsMzJfLMUVvOLTMk2WMoLu+MJJggtkhwtB+Mu/MUIf4V6b+Tpe5449WtLvVr
M+QvY7LN5OffSO6Slsy9oisGTdfE39nC7eVRABEBAAG0N01pT3Jvc29mdCAoUmVs
ZWFzZSBzaWduaW5nKSA8bm9yZ2pkMlMbWljon9zb2Z0LnNvbT6JNTQEEwECI
AB4FAlYxWIwCGwMGCwkIBwMCAxUIAwMWAgECHgECF4AACgkQ6z6Urb45Ro+F9gf/
dLT2900wvWEgV7LlMgrtOzx79W/FbwWLOkKoD9sdzhNhWlF8D5teL/t5TUM6TZ3B
EMboOJwr7W5JLLFwutDtFI9jz4kvdT86JuwD6M8gn4MBLdt5b5dJEtXrzwtlB/fe
6tyfMT1E/FrJfgMLUO9M4TIJko0OhMJoyFp0Z6TW5L0XnkAo8MltTKDfR7q4utI8
4oBu4nJZMZbxIO6r8Cot+xxuJkpOTIpr4edpOKmECM6x55aT2gVnscbN8FE19MN9
nFsBxyK4wW4MvMned2qldBFTtpgzFhqB2gu0J5ryti95bMr5mlTJd1T1XfMMno5D
xfn5Jwgy5BId4WvtOl05gw==
=zFfd
-----END PGP PUBLIC KEY BLOCK-----
```

Figure 47: Installing Dependencies (2)

```
azureuser@techstart-webvm:~$ sudo apt update
Hit:1 http://azure.archive.ubuntu.com/ubuntu noble InRelease
Hit:2 http://azure.archive.ubuntu.com/ubuntu noble-updates InRelease
Hit:3 http://azure.archive.ubuntu.com/ubuntu noble-backports InRelease
Hit:4 http://azure.archive.ubuntu.com/ubuntu noble-security InRelease
Get:5 https://packages.microsoft.com/ubuntu/24.04/prod noble InRelease [3600 B]
Get:6 https://packages.microsoft.com/ubuntu/24.04/prod noble/main armhf Packages [11.2 kB]
Get:7 https://packages.microsoft.com/ubuntu/24.04/prod noble/main amd64 Packages [73.9 kB]
Get:8 https://packages.microsoft.com/ubuntu/24.04/prod noble/main all Packages [643 B]
Get:9 https://packages.microsoft.com/ubuntu/24.04/prod noble/main arm64 Packages [55.0 kB]
Fetched 140 kB in 1s (184 kB/s)
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
All packages are up to date.
```

Figure 48: Installing Dependencies (3)



Figure 49: Installing Dependencies (4)



Figure 50: ODBC Driver Status

I then set up the necessary files in the app directory to create the website, with 'app.py' for the web application, 'login.html' for the login page, and 'dashboard.html' for the staff's dashboards (Fig. 51).

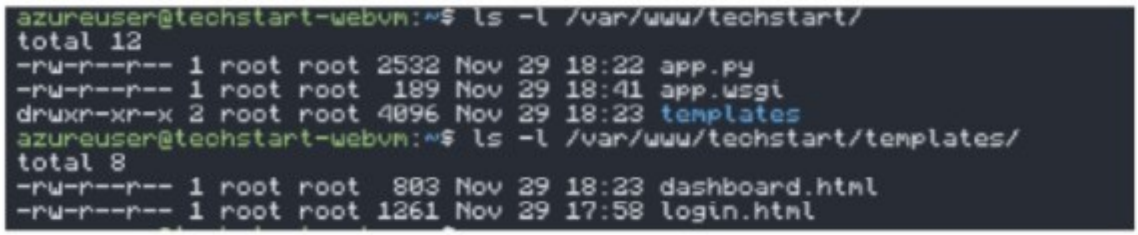


Figure 51: App Directory

I then proceeded to create and populate the mentioned database for all staff. With an entity named 'staff_directory' (Fig. 52) in which I put in staff details such as their staff ID, email, the

hashed password, full name, role, and last login (with the GETDATE() function) (Fig. 53). Confirmation of the entity's creation and population can be seen with the 'SELECT * FROM staff_directory' command (Fig. 54).

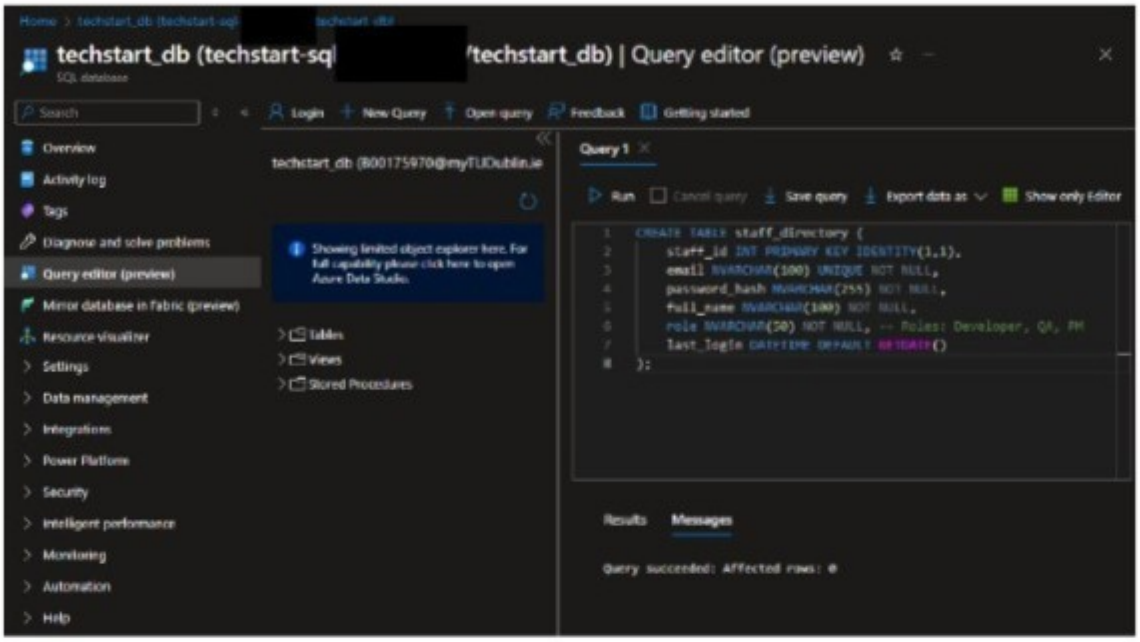


Figure 52: DB Creating Staff Table

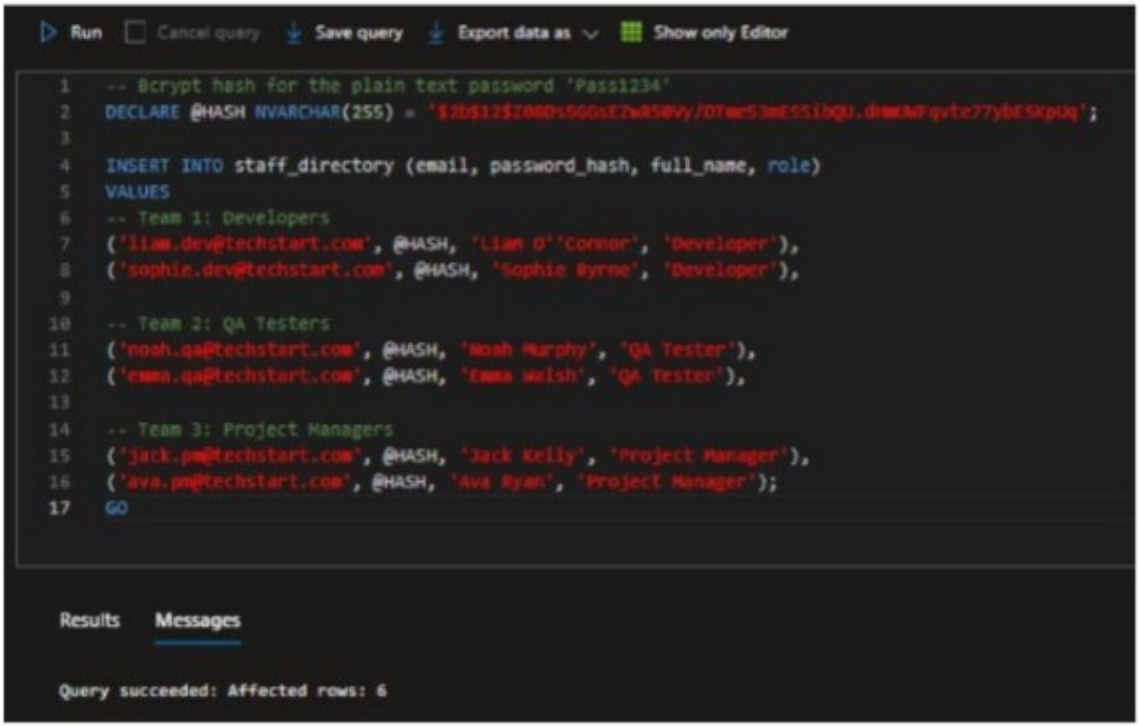


Figure 53: DB Adding Staff

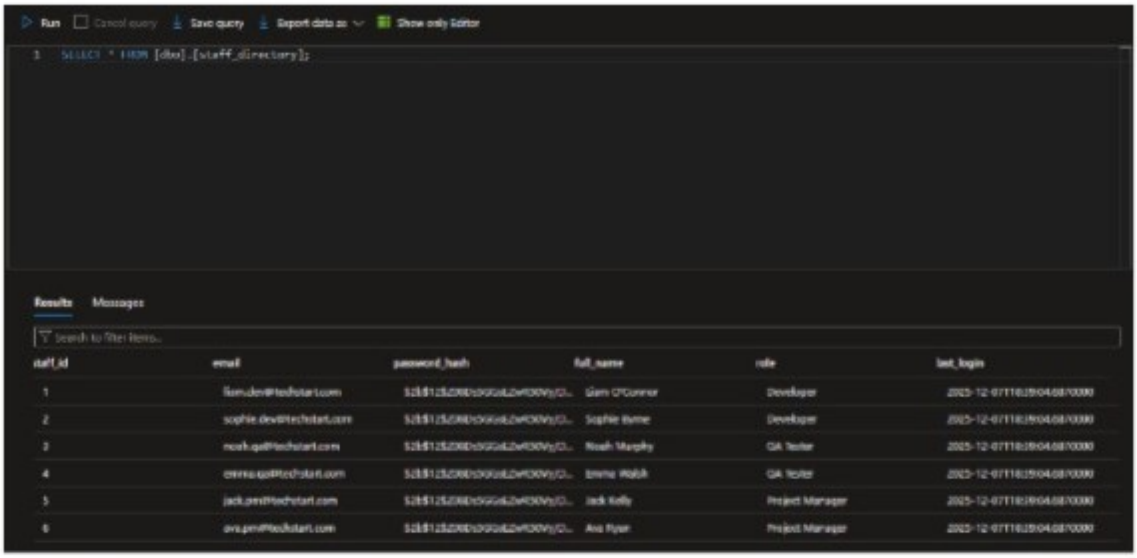


Figure 54: DB Staff Output

I also created a second table for all projects the staff are working on (Fig. 55-57).

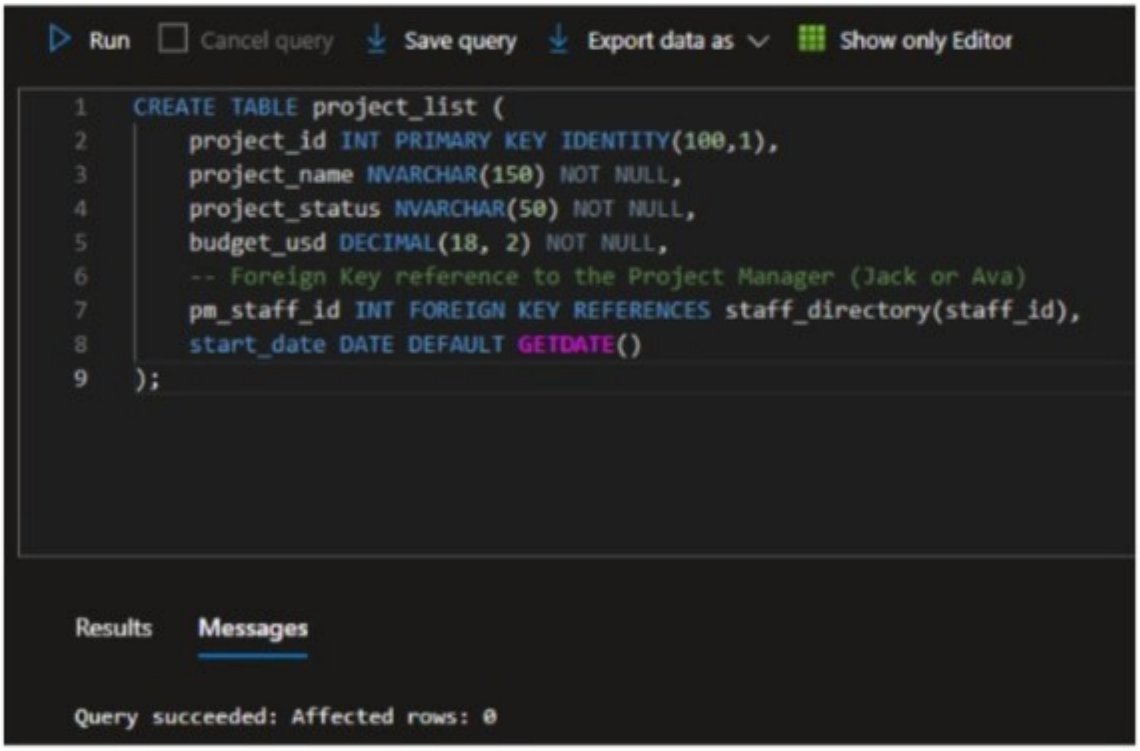


Figure 55: DB Creating Projects Table

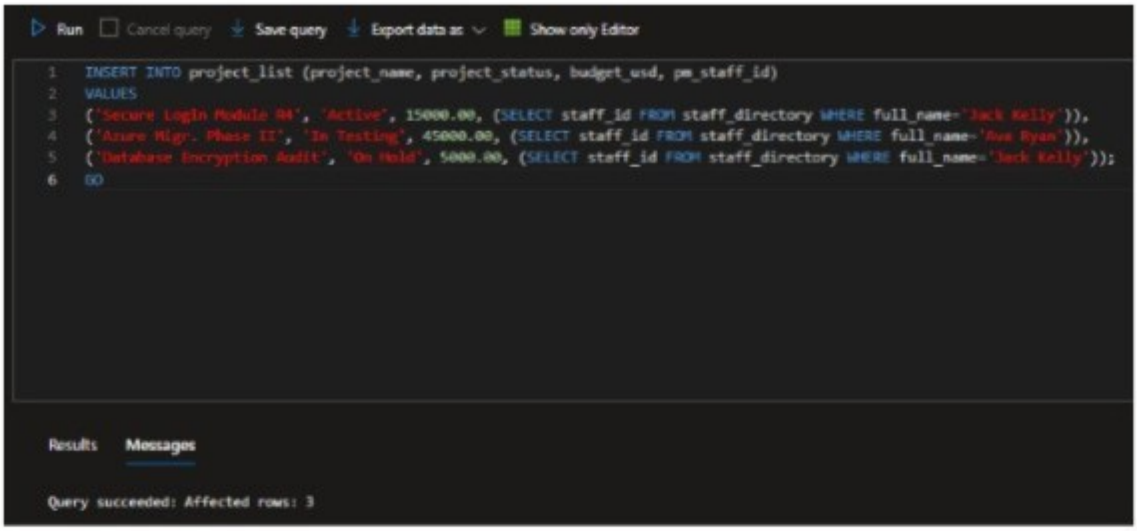


Figure 56: DB Adding Projects

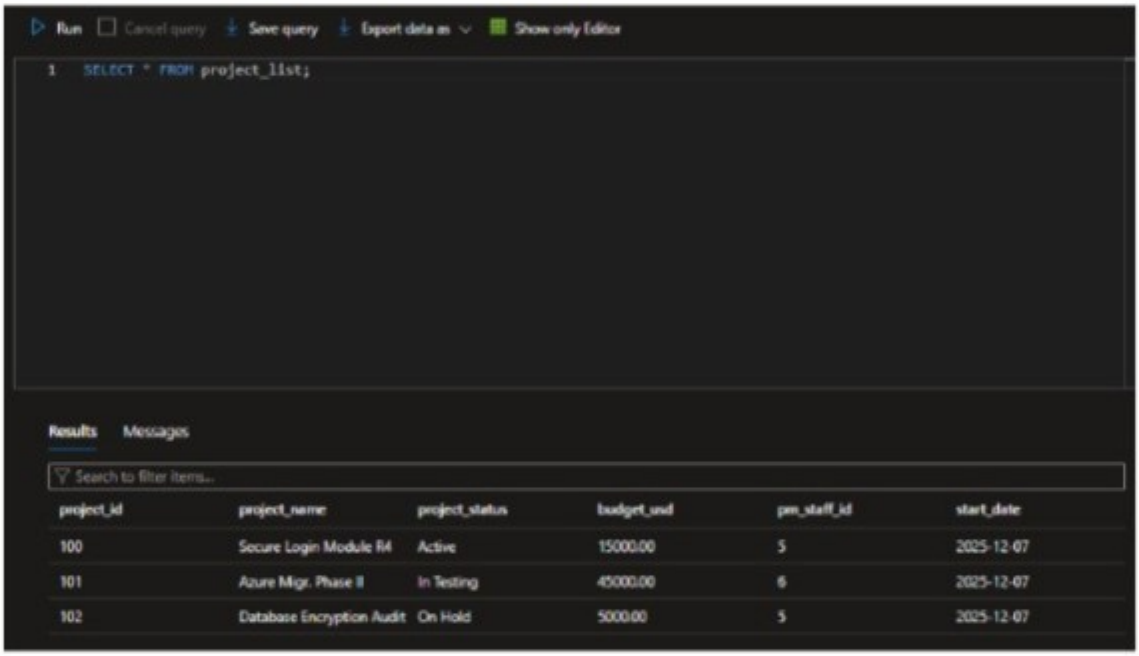


Figure 57: DB Projects Output

I then installed Apache because it acts as the web server software, handling requests from the browser and delivering the website's content (Fig. 58). For this I needed 2 files: 'techstart-ssl.conf' (Fig.59) for configuring Apache using port 443 (HTTPS), and 'app.wsgi' (Fig. 60) which serves as the entry point and translator between the Python web application and the web server software (Apache). As mentioned, I used HTTPS instead of HTTP since it encrypts the data in between the browser and server.

```
azureuser@techstart-webvm:~$ sudo nano /etc/apache2/sites-available/techstart-ssl.conf
azureuser@techstart-webvm:~$ sudo a2enmod ssl
Considering dependency mime for ssl:
Module mime already enabled
Considering dependency socache_shmcb for ssl:
Enabling module socache_shmcb.
Enabling module ssl.
See /usr/share/doc/apache2/README.Debian.gz on how to configure SSL and create self-signed certificat
es.
To activate the new configuration, you need to run:
  systemctl restart apache2
azureuser@techstart-webvm:~$ sudo a2ensite techstart-ssl.conf
Enabling site techstart-ssl.
To activate the new configuration, you need to run:
  systemctl reload apache2
azureuser@techstart-webvm:~$ sudo systemctl restart apache2
```

Figure 58: Installing Apache

```
  GNU nano 7.2     /etc/apache2/sites-available/techstart-ssl.conf
<VirtualHost *:80>
    # Redirect HTTP to HTTPS:
    ServerName 20.68.195.130
    Redirect permanent / https://20.68.195.130/
</VirtualHost>

<VirtualHost *:443>
    ServerAdmin admin@techstart.com

    # SSL Configuration:
    SSLEngine on
    SSLCertificateFile /etc/ssl/certs/apache-selfsigned.crt
    SSLCertificateKeyFile /etc/ssl/private/apache-selfsigned.key

    # Define the WSGI DAEMON Process
    WSGIDaemonProcess techstart user=www-data group=www-data threads=5

    # Alias the Root URL (/) to the WSGI file
    WSGIScriptAlias / /var/www/techstart/app.wsgi

    # Directory Permissions
    <Directory /var/www/techstart>
        WSGIProcessGroup techstart
        WSGIApplicationGroup %{GLOBAL}
        Require all granted
    </Directory>

    # Logging
    ErrorLog ${APACHE_LOG_DIR}/techstart-error.log
    CustomLog ${APACHE_LOG_DIR}/techstart-access.log combined
</VirtualHost>
```

Figure 59: Configuring Apache

```
  GNU nano 7.2     /var/www/techstart/app.wsgi
#!/usr/bin/env python3

import sys
import logging

logging.basicConfig(stream=sys.stderr)

sys.path.insert(0, '/var/www/techstart')

from app import app as application
```

Figure 60: WSGI File

And finally, with the whole setup complete, the login page displays (Fig. 61), and the dashboards show when logging in with the correct credentials (Fig. 62-64).

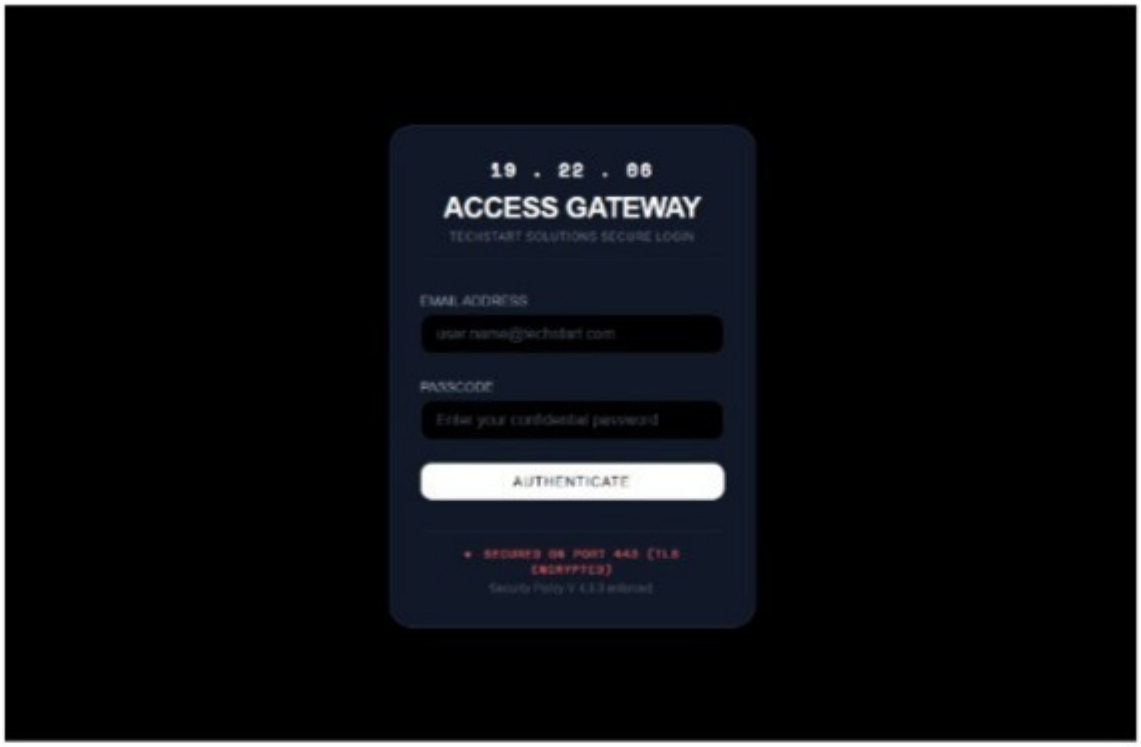


Figure 61: Login Page

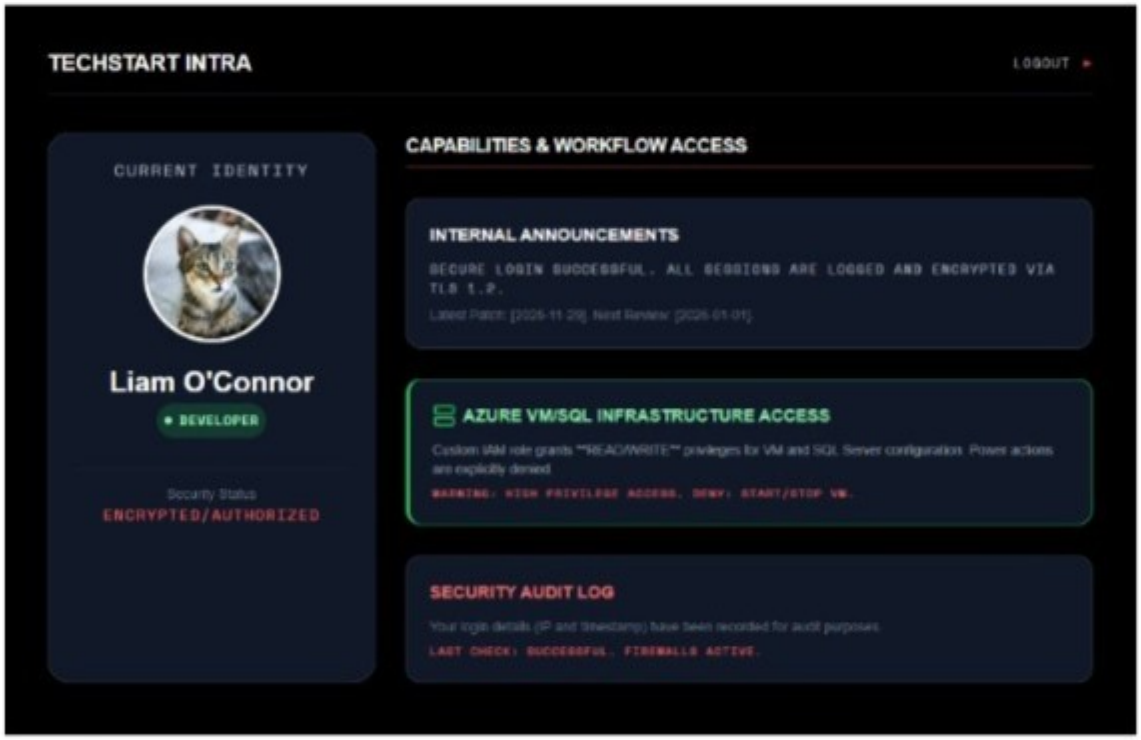


Figure 62: Dashboard Page (1)

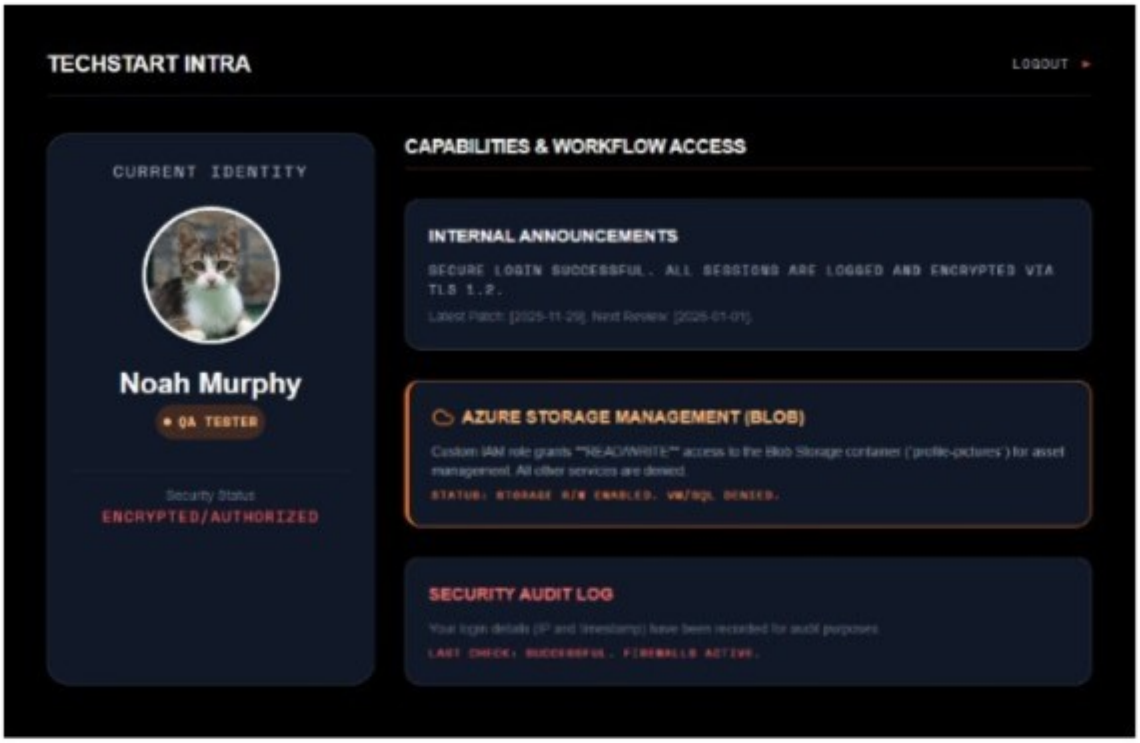


Figure 63: Dashboard Page (2)

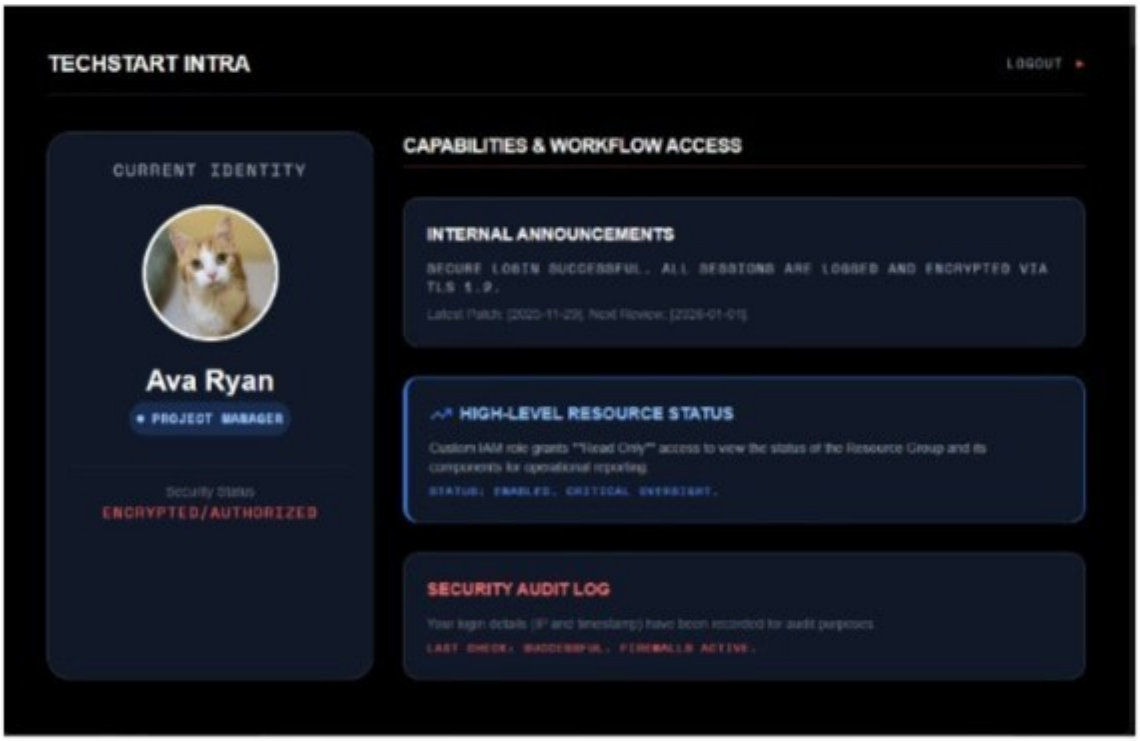


Figure 64: Dashboard Page (3)

Conclusion

In this project I created a cloud infrastructure for TechStart Solutions, in which I deployed a working and complete web application in a protected environment.

The design strictly followed the Principle of Least Privilege and Defence in Depth. The Principle of Least Privilege was demonstrated with network segmentation (web & database subnets), access controls (specific permissions for employees), and the key-based SSH authentication. The Defence in Depth was demonstrated by using security controls across the system, such as a network perimeter (Network Resource Group), host perimeter (Uncomplicated Firewall), system hardening (unattended-upgrades, PAM, Fail2Ban, password encryption, and harden.sh), and data protection (HTTPS for Encryption in Transit, and Azure TDE with Blob for encryption At Rest).

To summarise, the web application is accessible through HTTPS, and the backend SQL database and Blob storage which are securely connected and populated with all mandatory security requirements are up and working.